# EECS708: Machine Learning

Classification – part 2

Dr. Ioannis Patras
School of EECS

Slide thanks: Dr. Tim Hospedales

# Course Context

- Supervised Learning
  - (Linear) regression
  - (Linear) Classifiers and Logistic Regression
  - Neural Networks
- Unsupervised
  - Clustering
  - Density Estimation
  - Dimensionality reduction (partial)
- Advanced topics
  - Deep Learning, Convolutional Neural Networks
  - Ensemble Learning

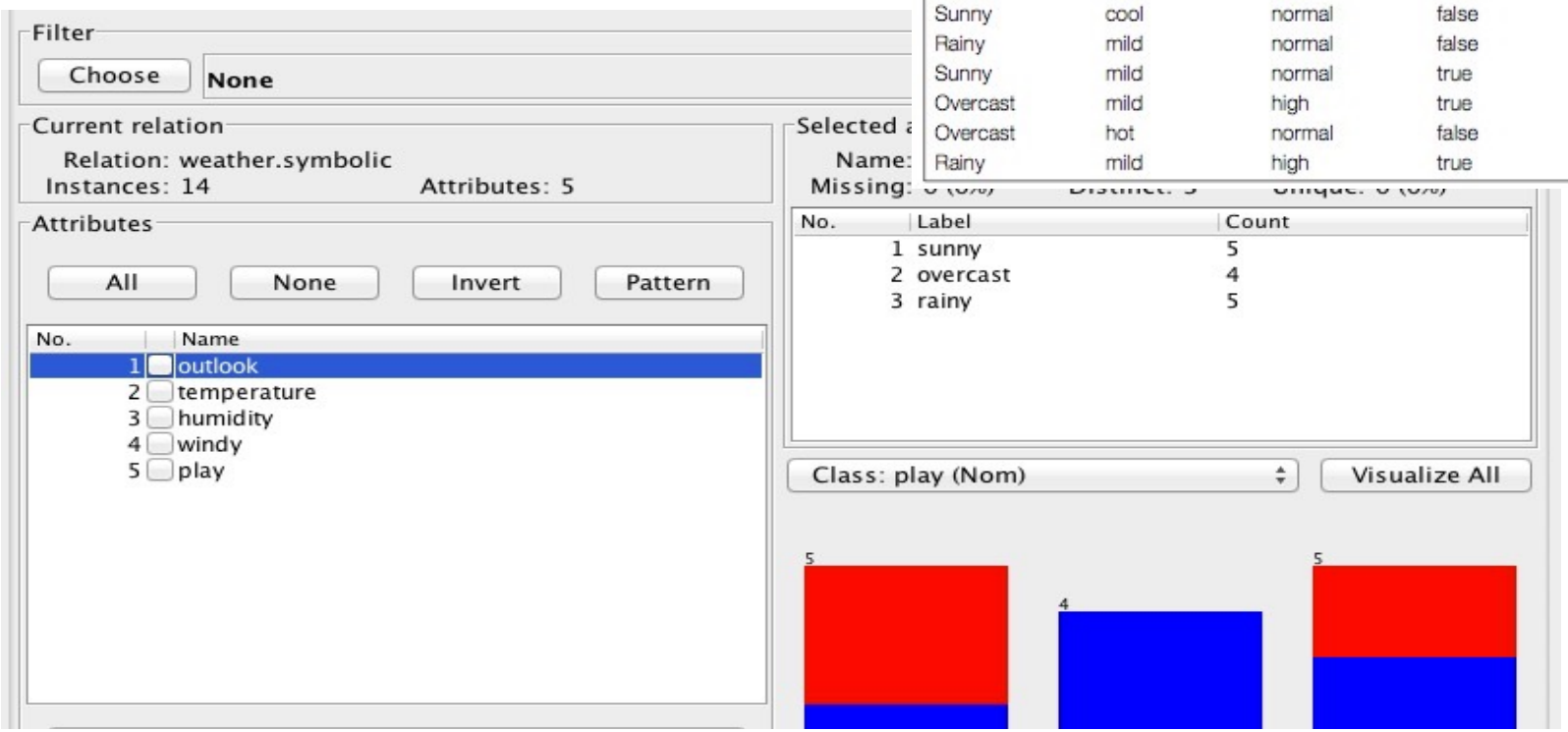
# Classification: Overview

- Decision Trees
- Naïve Bayes
- Practical Issues and performance metrics

# Decision Trees:
# Play tennis dataset

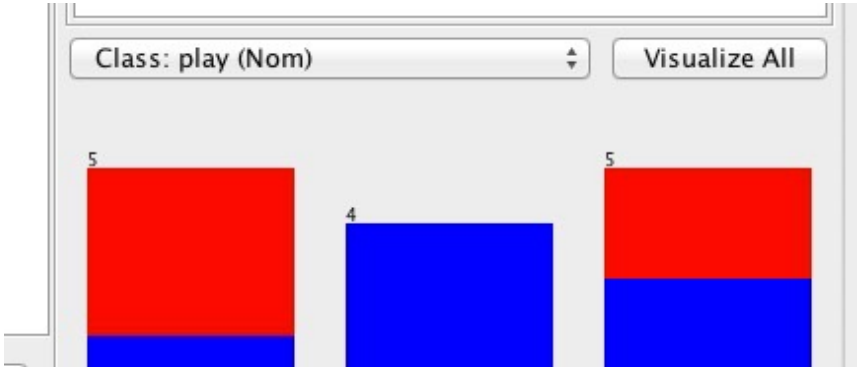
**Table 1.2** Weather Data

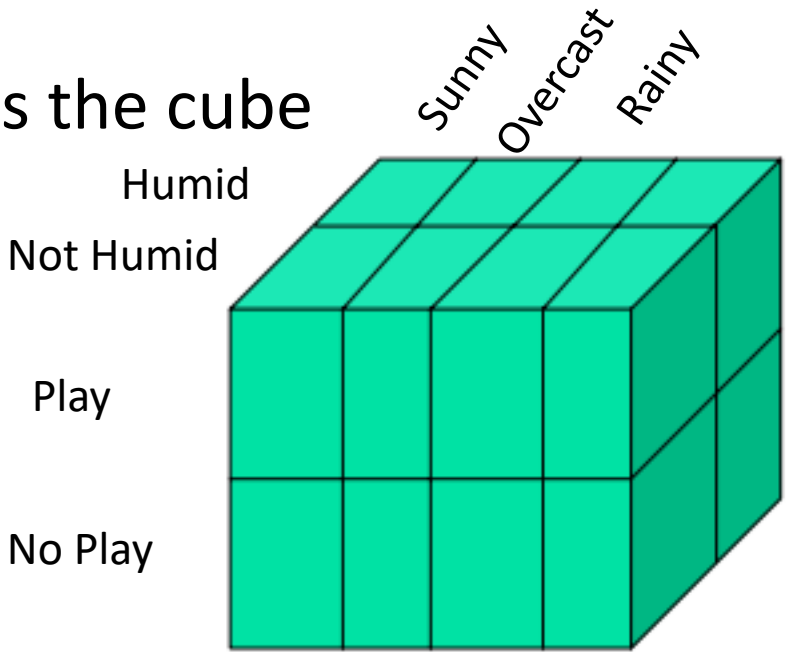
| Outlook | Temperature | Humidity | Windy | Play |
|---|---|---|---|---|
| Sunny | hot | high | false | no |
| Sunny | hot | high | true | no |
| Overcast | hot | high | false | yes |
| Rainy | mild | high | false | yes |
| Rainy | cool | normal | false | yes |
| Rainy | cool | normal | true | no |
| Overcast | cool | normal | true | yes |
| Sunny | mild | high | false | no |
| Sunny | cool | normal | false | yes |
| Rainy | mild | normal | false | yes |
| Sunny | mild | normal | true | yes |
| Overcast | mild | high | true | yes |
| Overcast | hot | normal | false | yes |
| Rainy | mild | high | true | no |



# Decision Trees: Contingency Tables

- For every combination of attributes, record how frequently it occurs
- Check the cube to predict new data
  - Would be slow
  - Decision tree can compress the cube



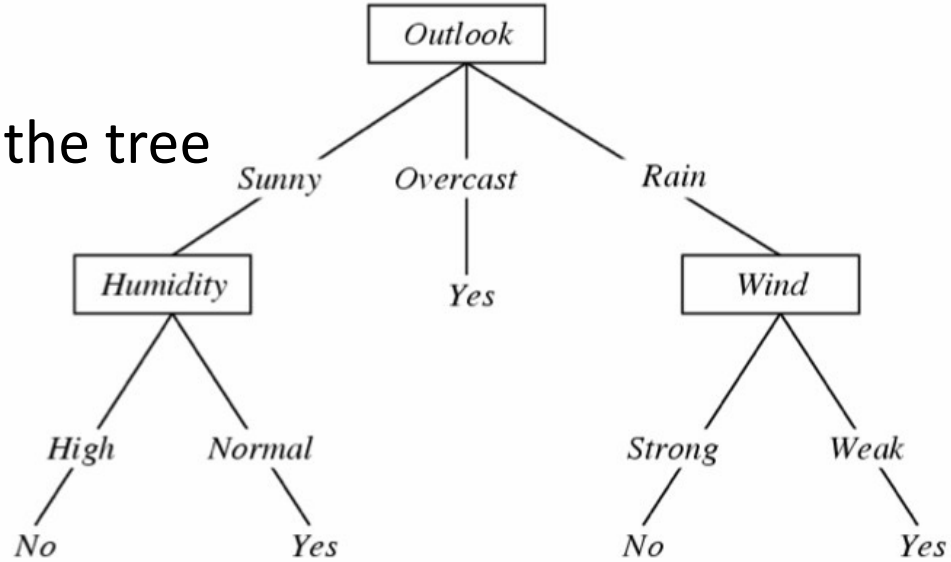
# Decision Trees: Model Structure & Test time procedure

- Internal nodes:
  - Test the value of a particular attribute: Equality / Inequality.
  - Branch according to the result
- Leaf nodes:
  - Specify the class f(x)
- Test time:

  Classify x* by sending it down the tree

**Table 1.2** Weather Data

| Outlook | Temperature | Humidity | Windy | Play |
|---|---|---|---|---|
| Sunny | hot | high | false | no |
| Sunny | hot | high | true | no |
| Overcast | hot | high | false | yes |
| Rainy | mild | high | false | yes |
| Rainy | cool | normal | false | yes |
| Rainy | cool | normal | true | no |
| Overcast | cool | normal | true | yes |
| Sunny | mild | high | false | no |
| Sunny | cool | normal | false | yes |
| Rainy | mild | normal | false | yes |
| Sunny | mild | normal | true | yes |
| Overcast | mild | high | true | yes |
| Overcast | hot | normal | false | yes |
| Rainy | mild | high | true | no |

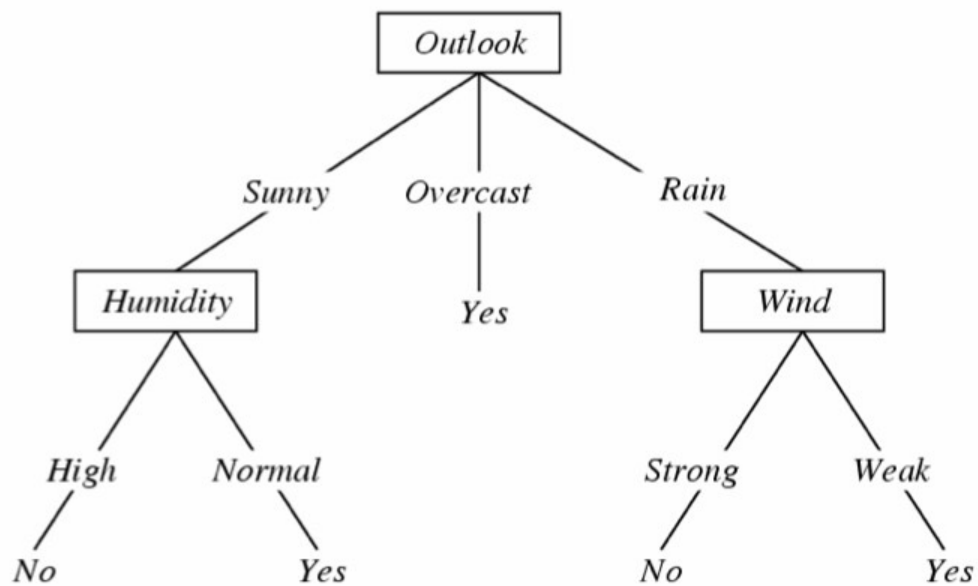


# Decision Tree: How to Grow/Train

- What algorithm do you think can construct a tree from data?
  - Hint: It's recursive.
- Suppose you have a magic pick-best attribute function?

**Table 1.2** Weather Data

| Outlook | Temperature | Humidity | Windy | Play |
|---|---|---|---|---|
| Sunny | hot | high | false | no |
| Sunny | hot | high | true | no |
| Overcast | hot | high | false | yes |
| Rainy | mild | high | false | yes |
| Rainy | cool | normal | false | yes |
| Rainy | cool | normal | true | no |
| Overcast | cool | normal | true | yes |
| Sunny | mild | high | false | no |
| Sunny | cool | normal | false | yes |
| Rainy | mild | normal | false | yes |
| Sunny | mild | normal | true | yes |
| Overcast | mild | high | true | yes |
| Overcast | hot | normal | false | yes |
| Rainy | mild | high | true | no |



# Decision Tree: How to Grow/Train

Recursive Algorithm:

- Grow(T)
  - if All y=0, return Leaf(0)
  - elseif All y=1, return Leaf(1)
  - else
    - $x_j$ = ChooseBestAttribute(T)
    - T0 = <x,y> in T with $x_j$=0
    - T1 = <x,y> in T with $x_j$=1
    - Return Node($x_j$, Grow(T0), Grow(T1))

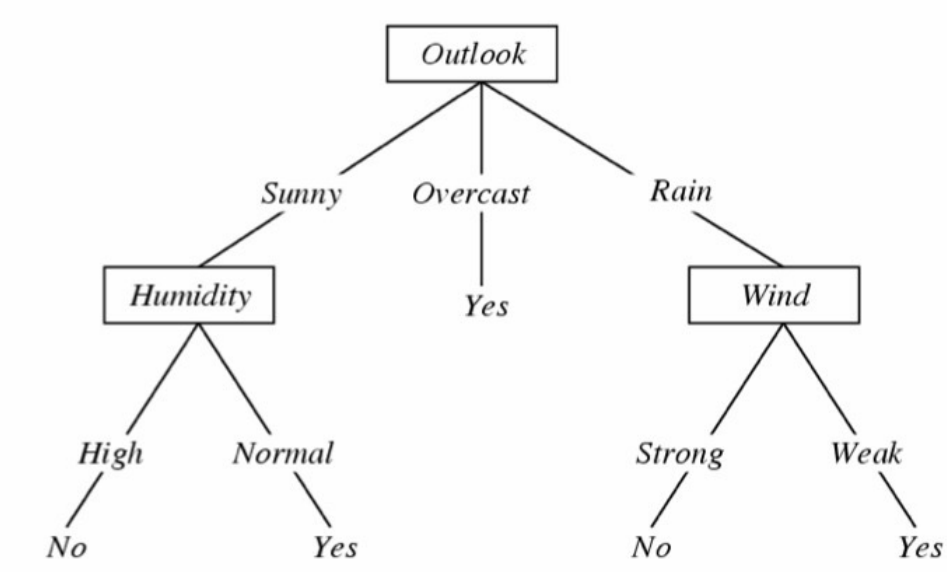



**Table 1.2** Weather Data

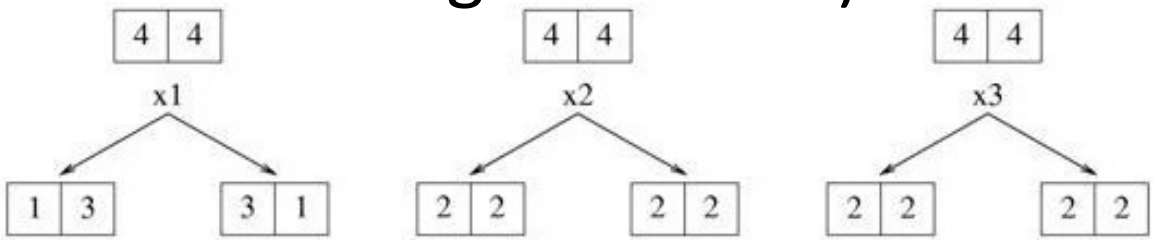
| Outlook | Temperature | Humidity | Windy | Play |
|---|---|---|---|---|
| Sunny | hot | high | false | no |
| Sunny | hot | high | true | no |
| Overcast | hot | high | false | yes |
| Rainy | mild | high | false | yes |
| Rainy | cool | normal | false | yes |
| Rainy | cool | normal | true | no |
| Overcast | cool | normal | true | yes |
| Sunny | mild | high | false | no |
| Sunny | cool | normal | false | yes |
| Rainy | mild | normal | false | yes |
| Sunny | mild | normal | true | yes |
| Overcast | mild | high | true | yes |
| Overcast | hot | normal | false | yes |
| Rainy | mild | high | true | no |

# Grow/Train: How to choose best attribute?

- Pick the attribute that greedily maximizes accuracy?
  - In this example, x1
- j = ChooseBestAttribute(T)
  - Choose j to minimize
  - #Examples <x,y> in T0 with y!=0

  +
  - #Examples <x,y> in T1 with y!=1
- (Actually, minimize information gain instead)

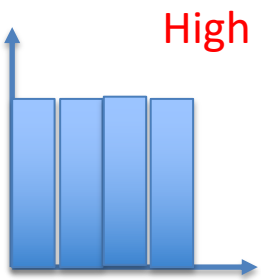
| $x_1$ | $x_2$ | $x_3$ | $y$ |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 |

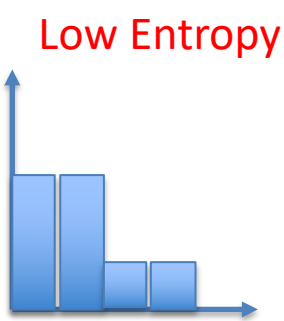


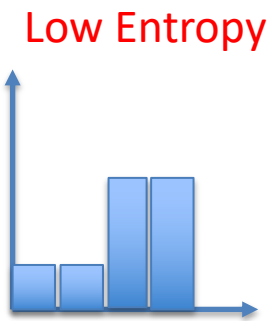
# Entropy and Information Gain

Entropy of a random variable Y

$$H(Y) = -\sum_y p(y)\log(p(y)))$$



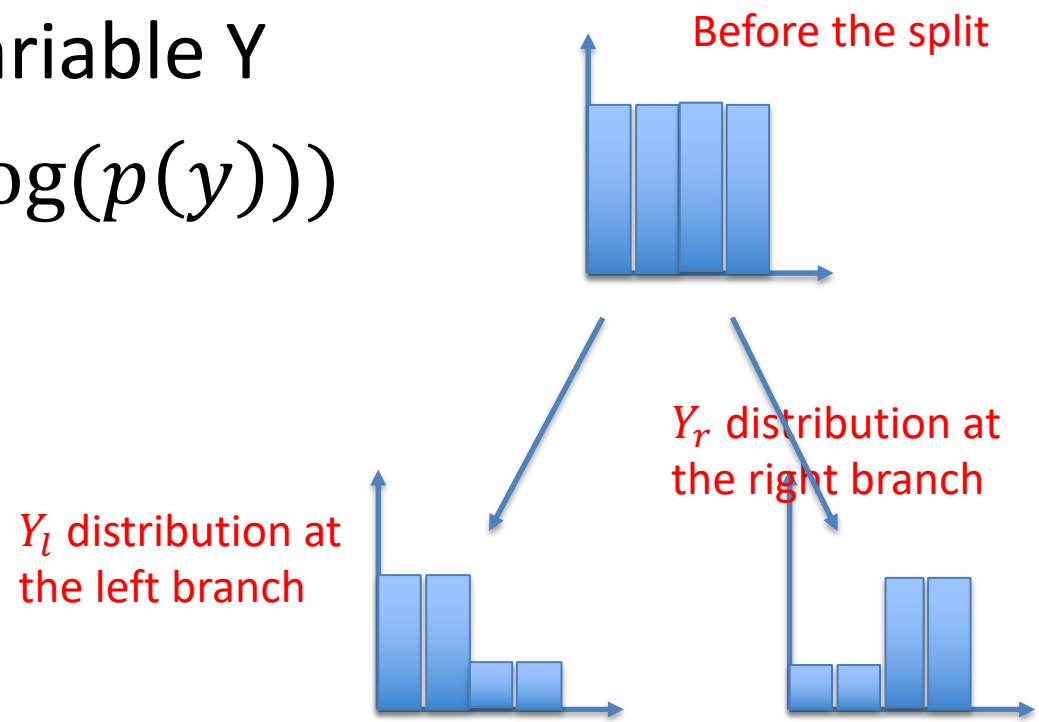




A measure of how uniform/peaked a distribution is

# Information Gain in decision trees

Entropy of a random variable Y

$$H(Y) = -\sum_y p(y)\log(p(y)))$$




Information gain of Y given a split A={l,r}

$$IG(Y|A) = H(Y) - \sum p_a H(Y_a)$$

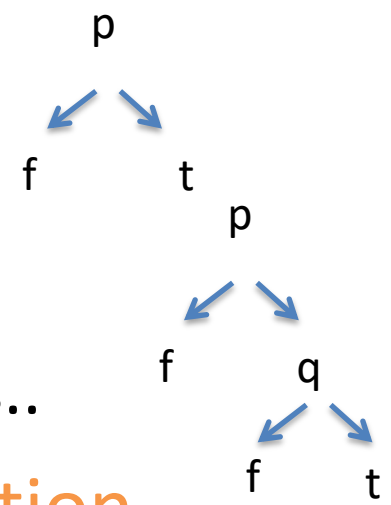
$p_a$ proportion of the data in branch $a \in \{l, r\}$

# Training with non-boolean features

- Nominal
  - Test one value versus all the others (Outlook=Sunny)
  - Group into disjoint subsets. (Postcode = W1)
- Continuous
  - Threshold inequality $x_j > th$

# Decision Tree: What can they represent? (Nominal Data)

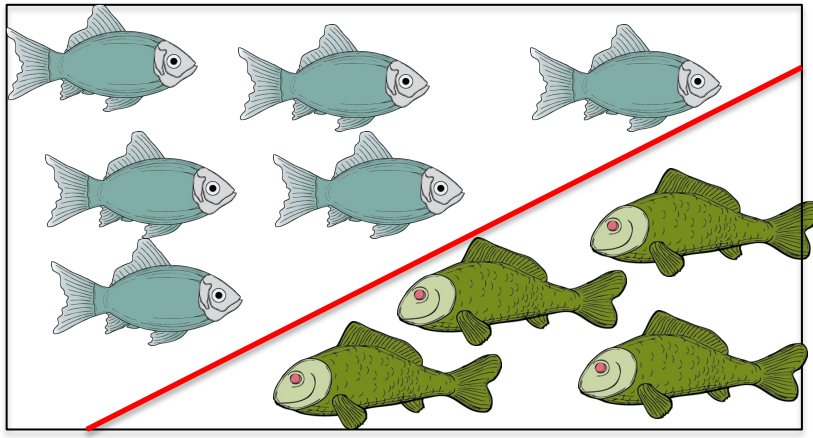
- Depth 1 tree
  - Any Boolean function of 1 feature..
- Depth 2 tree
  - Any Boolean function of two features..
- DT can represent any boolean function
  - (But worst case 2^N leaves)

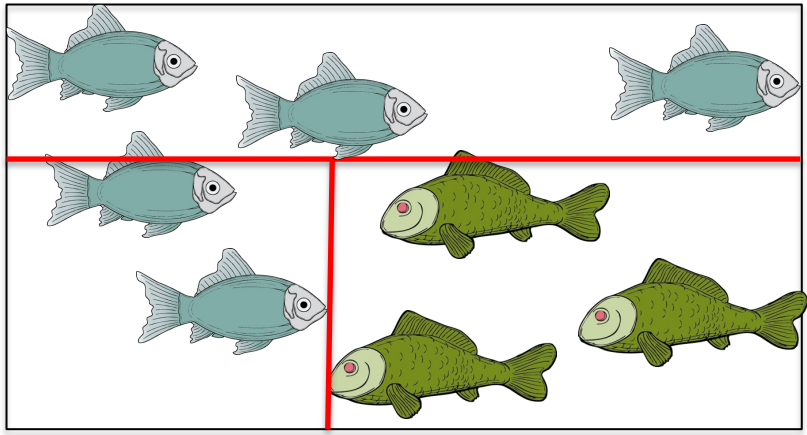



| p | q | p ∧ q |
|---|---|---|
| T | T | T |
| T | F | F |
| F | T | F |
| F | F | F |

# Decision Tree: What can they represent? (Continuous Data)

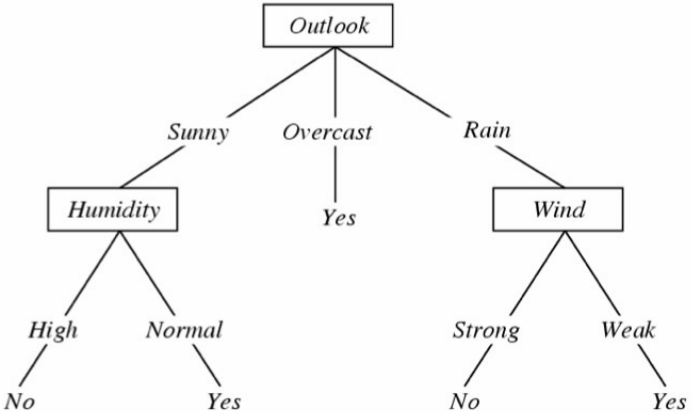
- If Length > L1
  - Then Salmon
- Else
  - If Lightness >L2
    - Then Cod
  - Else
    - Then Salmon
- Represent:
  - **Axis parallel cuts.**
  - Can approximate but not exactly represent diagonal boundaries.
  - Can become arbitrarily complex with enough data





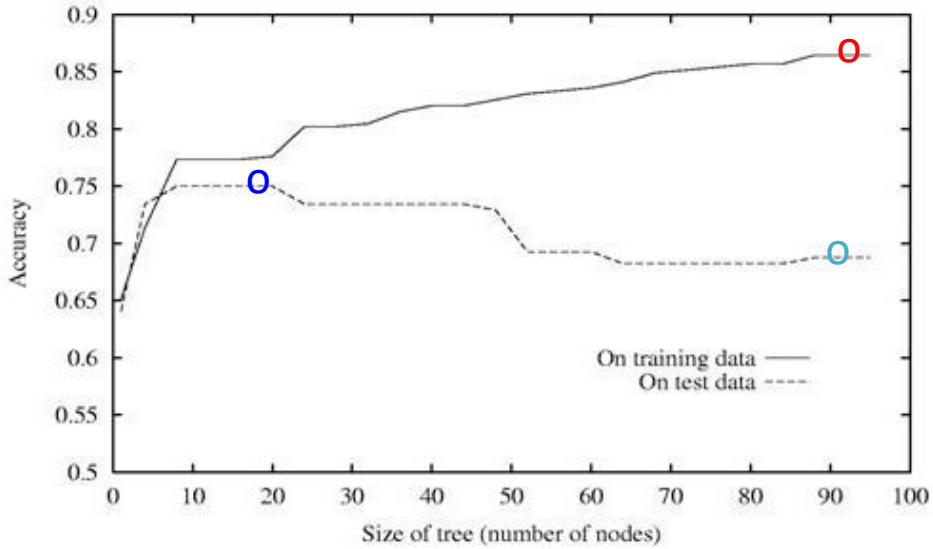
# Decision Tree: Over-fitting & Regularization

- Suppose one unusual day: [*Sunny*, *Hot*, *Normal*, *Strong*, *Play*=*No*]
  - What happens to tree?
  - New (noisy) nodes will be grown under Sunny-Normal-…



# Decision Tree: Over-fitting

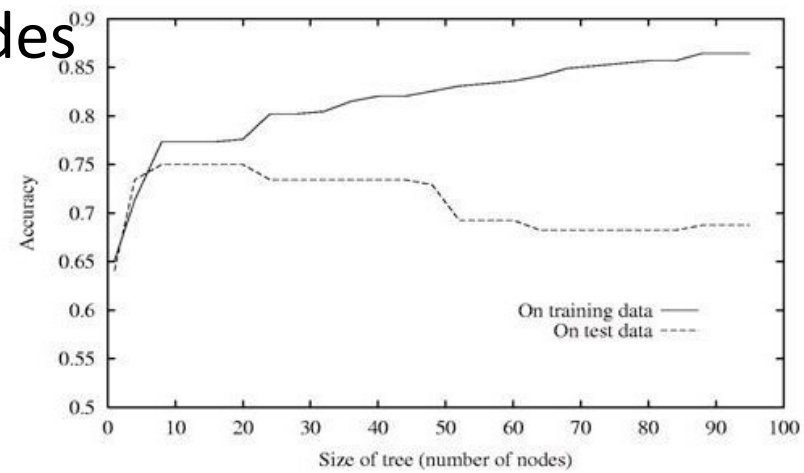
- Overfitting, formally:
  - Train Error (known): $E(M, D_{train})$
  - Future Error (unknown): $E(M, D_{all})$
  - Overfit model M if:
    - If there is some other model M'
    - $E(M, D_{train}) < E(M', D_{all})$
    - $E(M, D_{all}) > E(M', D_{all})$



# Decision Tree: Regularization

Avoiding Over-fitting for a decision tree. Ideas?

1. Grow full tree then prune
   - How to guide pruning?
     - Measure performance on train data?
     - Measure performance on validation data?
2. Add regularizer to split objective
   - $x_j$ = ChooseBestAttribute(T)
   - If error improvement < λ* #nodes
     - Then skip
   - (Determine λ by validation)



# Decision Tree: Summary

- Decision Tree Classifier:
  - Representation: Tree
  - Evaluation: Accuracy
  - Train: Greedy, Recursive
  - Test: Traverse tree
  - Prevent overfit:
    - Regularize on # of nodes, or
    - Pruning

Properties:

- Good:
  - Mixed-type data (no 1-of-N encode!)
  - High dimensions
- Classification at test time can take $< O(d)$!
  - CF: NN: $O(dn)$, MaxEnt: $O(d)$
- Frequently used in industry
- May be Interpretable
- Optimal tree is NP-complete
  - Practical trees are not optimal, but good enough
- Some pathological problems can't be represented as trees

# Classification: Overview

- Decision Trees
- Naïve Bayes
  - Bayes Theorem
  - ML fitting
- Practical Issues

# Naïve Bayes – Bayes Theorem Recap

- Bayes Theorem
  - P(A) "Prior probability of A"
  - P(B|A) "Probability of B given A"

$$p(A \mid B) = \frac{p(B \mid A)p(A)}{p(B)}$$

$$p(B \mid A)p(A) = p(A,B)$$

$$p(A) = \sum_B p(A,B)$$

**Does patient have cancer or not?**

A patient takes a lab test and the result comes back positive. The test returns a correct positive result in only 98% of the cases in which the disease is actually present, and a correct negative result in only 97% of the cases in which the disease is not present. Furthermore, 0.008 of the entire population have this cancer.

# Naïve Bayes – Bayes Theorem Recap

- Bayes Theorem:
  - P(H) "Prior probability of hypothesis H"
  - P(D|H) "Probability of data D given hypothesis H"

$$p(H \mid D) = \frac{p(D \mid H)p(H)}{p(D)}$$

**Does patient have cancer or not?**

A patient takes a lab test and the result comes back positive. The test returns a correct positive result in only 98% of the cases in which the disease is actually present, and a correct negative result in only 97% of the cases in which the disease is not present. Furthermore, 0.008 of the entire population have this cancer.

$$p(D \mid H)p(H) = p(D,H)$$

$$p(D) = \sum_{H} p(D,H)$$

# Naïve Bayes – Bayes Theorem Recap

- Bayes Theorem.
  - Hypothesis = {C,!C}, Data = {+,-}
  - P(C) = 0.008
  - P(!C) = 0.992
  - P(+|C) = 0.98
  - P(-|C)=0.02
  - P(+|!C)=0.03
  - P(-|!C)=0.97

$$p(H \mid D) = \frac{p(D \mid H)p(H)}{p(D)}$$

$$p(D \mid H)p(H) = p(D,H)$$

$$p(D) = \sum_{H} p(D,H)$$

$$p(C \mid +) = \frac{0.98 \times 0.008}{0.98 \times 0.008 + 0.03 \times 0.992}$$

$$p(C \mid +) \approx 0.2$$

**Does patient have cancer or not?**

A patient takes a lab test and the result comes back positive. The test returns a correct positive result in only 98% of the cases in which the disease is actually present, and a correct negative result in only 97% of the cases in which the disease is not present. Furthermore, 0.008 of the entire population have this cancer.

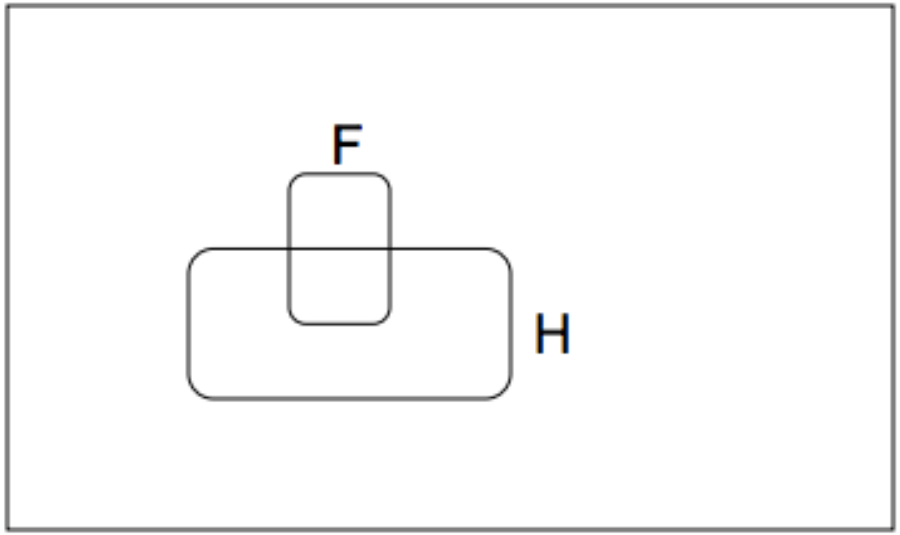
# Bayes Theorem - Visualization

- Set interpretation
  - P(A) is size of set A in the world
  - P(A,B) is the size of the intersection of set A&B
  - P(A|B) is the fraction of the space where B is true that A is also true

H = “Have a headache”
F = “Coming down with Flu”

P(H) = 1/10
P(F) = 1/40
P(H|F) = 1/2

“Headaches are rare and flu is rarer, but if you’re coming down with ‘flu there’s a 50/50 chance you’ll have a headache.”



# Bayes Theorem - Visualization

- Set interpretation
  - P(A) is size of set A in the world
  - P(A,B) is the size of the intersection of set A&B
  - P(A|B) is the fraction of the space where B is true that A is also true

H = "Have a headache"
F = "Coming down with Flu"

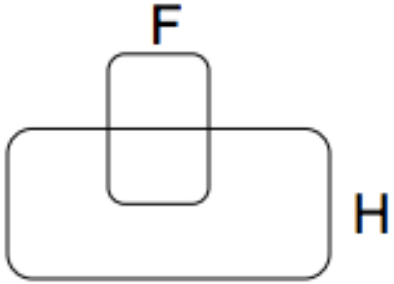
P(H) = 1/10
P(F) = 1/40
P(H|F) = 1/2

Answer is 1/8th.

Good Reasoning?!

One day you wake up with a headache. You think: "Drat! 50% of flus are associated with headaches so I must have a 50-50 chance of coming down with flu"

Think fraction of H occupied by F
Not fraction of F occupied by H.



# From Bayes Theorem to Naïve Bayes

$$p(H \mid D) = \frac{p(D \mid H)p(H)}{p(D)}$$

$$p(D \mid H)p(H) = p(D,H)$$

$$p(D) = \sum_H p(D,H)$$

- Bayes Theorem:
  - P(H) "Prior probability of hypothesis H"
  - P(D|H) "Probability of data D given hypothesis H"
- What if we have two or more sources of data?

Recall: $\boxed{p(A,B) = p(A)p(B) \text{ iff independent}}$

- Then either

$$p(H \mid D_1, D_2) = \frac{p(D_1, D_2 \mid H)p(H)}{p(D_1, D_2)} \qquad p(H \mid D_1, D_2) = \frac{p(D_1 \mid H)p(D_2 \mid H)p(H)}{p(D_1, D_2)}$$

- Using the latter regardless is known as the "Naïve" Assumption

# From Bayes Theorem to Naïve Bayes

- A direct Bayesian classifier would have to model:
  - P("Viarga"=1, "Cheap"=1,..|Spam)=….
  - P("Viarga"=0, "Cheap"=1,..|Spam)=….
  - P("Viarga"=1, "Cheap"=0,..|Spam)=….
  - P("Viarga"=0, "Cheap"=0,..|Spam)=….
  - ….
  - P("Viarga"=1, "Cheap"=1,..|Ham)=….
  - P("Viarga"=0, "Cheap"=1,..|Ham)=….
  - P("Viarga"=1, "Cheap"=0,..|Ham)=….
  - P("Viarga"=0, "Cheap"=0,..|Ham)=….
- Clearly the table size, and hence data requirement is exponential in the size of the dictionary. Cf: $2^{80,000}$!

$$p(H \mid D_1, D_2) = \frac{p(D_1, D_2 \mid H)\, p(H)}{p(D_1, D_2)}$$

# Naïve Bayes Classifier

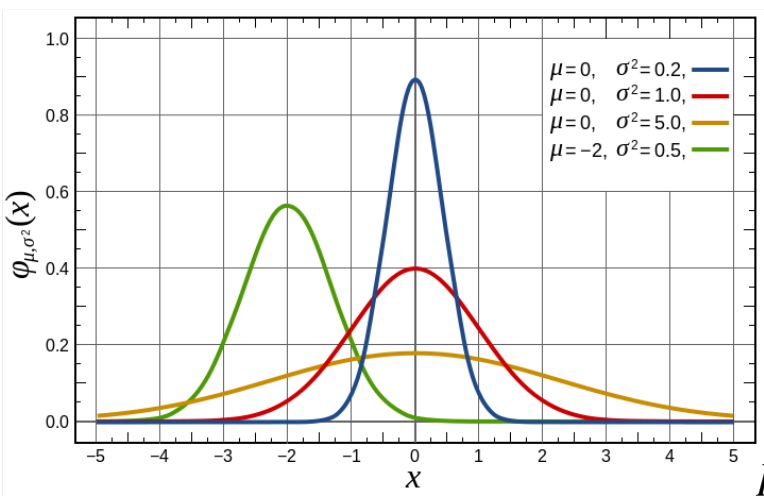
$$p(H \mid D_{1..N}) = \frac{\prod_i p(D_i \mid H) p(H)}{\prod_i p(D_i)}$$

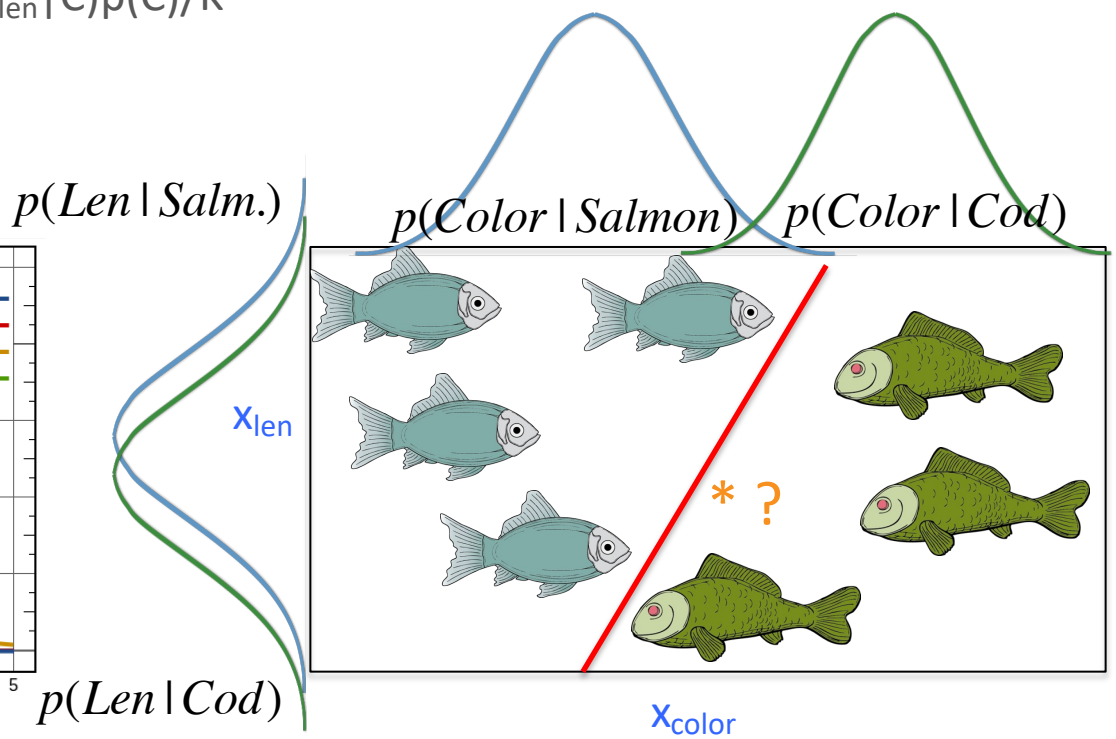
- Naïve Bayes spam classification
  - P("Viarga"|Spam)=90%
  - P("Viagra"|Ham) =5%
  - P("Cheap"|Spam)=60%
  - P("Cheap"|Ham) =30%
  - P(Spam)=10%
  - P(Ham)=90%
- P(Spam|Cheap) = p(C|S) p(S)/Z = 0.6*0.1/(0.6*0.1+0.3*0.9) = 18%
- P(Spam|Viagra) = p(V|S) p(S)/Z = 0.9*0.1/(0.9*0.1 + 0.05*0.9) = 67%
- P(Spam|Cheap,Viagra) = p(V|S)p(C|S)p(S)/Z
  - = 0.6*0.9*0.1/(0.6*0.9*0.1+0.3*0.05*0.9) = 80%

# Naïve Bayes Classifier: Continous Data

- For continuous data, often model p(D|H) as Gaussian
  - $P(S|x^*_{col},x^*_{len})=p(x^*_{col}|S)p(x^*_{len}|S)p(S)/K$
  - $P(C|x^*_{col},x^*_{len})=p(x^*_{col}|C)p(x^*_{len}|C)p(C)/K$

$$p(x\,|\,\mu,\sigma)=\frac{1}{\sigma\sqrt{2\pi}}\exp-\frac{1}{2\sigma^2}(x-\mu)^2$$



# Learning Naïve Bayes Classifier: Discrete

- To learn the NB classifier, need to fit probability distributions
  - Observe a coin with H,H,H,T,T.
    - p(Heads | Coin)=3/5, 60%, p(Tails | Coin)=2/5, 40%.
  - Roll Dice 60 times, observe: 12x1, 8x2, 11x3, 9x4, 14x5, 6x6
    - P(1|Dice) = 20%, …, p(6|Dice)=10%.
- This is called a binomial/multinomial distribution.
  - Parameter tells you the bias. [0.6, 0.4], [0.2,0.13,0.18,0.15,0.23.0.1]
- Find the parameter that maximizes the probability of the data
  - $W_{coin}$ = argmax p(D|$W_{coin}$)
  - ($N_j$ is counting number of outcomes of type j)

$$w_j = N_j / \sum_j N_j$$

# Learning Naïve Bayes Classifier: Discrete: Math and Pseudocode

- Find the parameter that maximizes the probability of the data
  - W = argmax p(D|W)

$$w_{jk} = N_{jk} / \sum_j N_{jk}$$

$$N_{jk} = \sum_i I(x_{ik} = j)$$

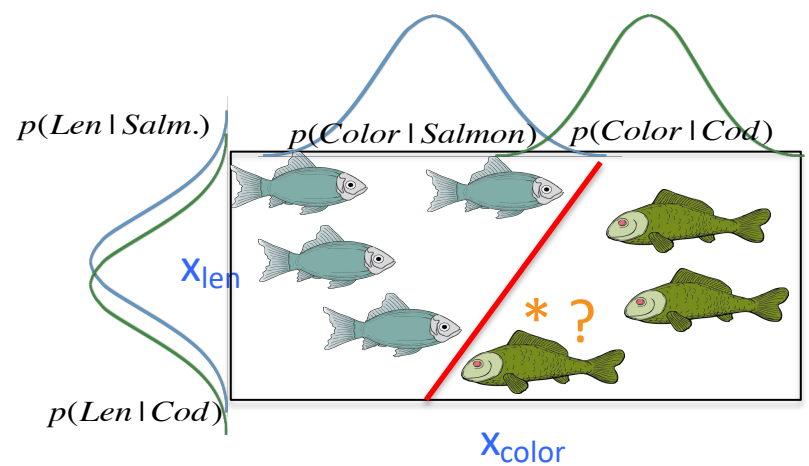
- Foreach attribute k
  - Foreach Data i
    - Foreach state j
      - N(j,k) +=1 if $x_{ik}$ = j
  - Make N(:,k) sum to 1

# Learning Naïve Bayes Classifier: Continuous

- To learn the NB classifier, independently find the parameter that maximizes the probability of the training data
- For Gaussian:
  - $\{\mu,\sigma\}=\text{argmax } p(D|\mu,\sigma)$
- $D=\{<x_l,x_c,\text{fish}>\}$
  - $=\{<0.1,0.3,\text{cod}>,<0.2,0.4,\text{cod}>,..$
  - $<0.3,0.2,\text{salm}>,<0.4,0.3,\text{salm}>\}$
- Then
  - Cod $\mu_{len}$ = (0.1+0.2+…)/N
  - Salmon $\mu_{len}$ = (0.3+0.4+….)/N, etc.

$$p(x\,|\,\mu,\sigma)=\frac{1}{\sigma\sqrt{2\pi}}\exp-\frac{1}{2\sigma^2}(x-\mu)^2$$

$$\mu=\frac{1}{N}\sum_i^N x \qquad \sigma^2=\frac{1}{N-1}\sum_i^N (x_i-\mu)^2$$



# Naïve Bayes Classifier: Over-fitting

- What if you had ten spams and no real emails with "viagra"?
  - Our parameter estimate equation:
  - P(Viagra|Spam)=10/10+0=100%
  - P(Viagra|Ham)=0/10+0=0%

$$p(x_j) = N_j / \sum_j N_j$$

- Now you get a long email from a friend that happens to mention Viagra:
  - The spam evidence from one "Viagra" overrides every other indication of ham from the email. (Multiply by zero)
  - How to fix?

$$p(H \mid D_{1..N}) = \frac{\prod_i p(D_i \mid H) p(H)}{\prod_i p(D_i)}$$

# Naïve Bayes Classifier: Regularization

- What if you had ten spams and no real emails with "Viagra"?
  - MLE Learning
    - P(Viagra|Spam)=10/10+0=100%
    - P(Viagra|Ham)=0/10+0=0%
  - Regularized Learning, λ=1
    - P(Viagra|Spam)=10+1/11+1=92%
    - P(Viagra|Ham)=1/11+1=8%
- Now, with enough positive evidence, an email could be Ham despite including Viagra.

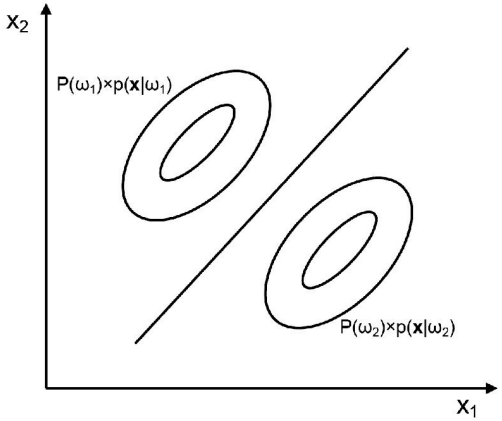
$$w_j = N_j / \sum_j N_j$$

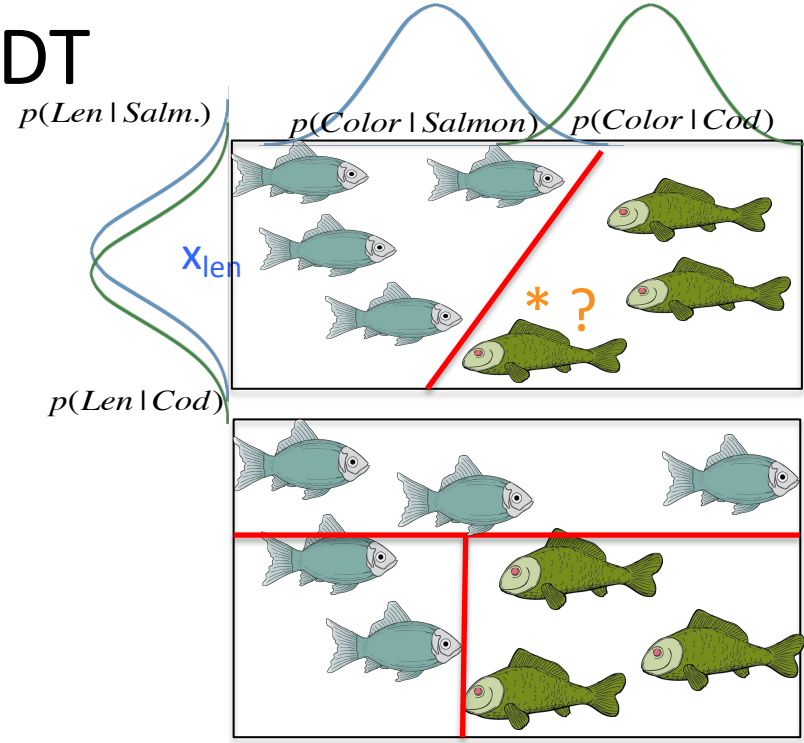
$$w_j = \left(N_j + \lambda\right) / \sum_j \left(N_j + \lambda\right)$$

$$p(H \mid D_{1..N}) = \frac{\prod_i p(D_i \mid H) p(H)}{\prod_i p(D_i)}$$

# NB Classifier: What can it classify?

- For continous data
  - Naïve Bayes models a line (or quadratic curve)
- For discrete data
  - Naïve Bayes models a line. (just like MaxEnt!)
- => Simpler boundary than DT



# Naïve Bayes Issues: Overconfidence

- Naïve assumption:
  - Counting each piece of evidence equally
  - Not exploiting attribute correlation

$$p(D_1, D_2 \mid H) = p(D_1 \mid H)\, p(D_2 \mid H)$$

$$p(D_{1..N} \mid H) = \prod_i p(D_i \mid H)$$

- You could attack a spam filter by listing all the fish species below your Viagra add…

# Naïve Bayes Classifier: Relation to MaxEnt

- Both classifiers have simple boundaries
- For data D={$y_i$,$x_i$}
- MaxEnt: $$\mathbf{w}^* = \operatorname{argmax} E_{MCL}(\mathbf{w}, D) = \prod_i p(y_i \mid \mathbf{x}_i, \mathbf{w})$$
- Naïve Bayes: $$\mathbf{w}^* = \operatorname{argmax} E_{ML}(\mathbf{w}, D) = \prod_i p(\mathbf{x}_i \mid y_i, \mathbf{w})$$
- NB Learning decouples the prior: $$p(H \mid D) = \frac{p(D \mid H)p(H)}{p(D)}$$
  - You can take your NB cancer classifier to Chernobyl and it will still work…
  - You can move your NB fish classifier from UK to Norway…
  - Your MaxEnt cancer classifier will have to re-train from scratch

# An Aside: Online Learning

- Sometimes you want to learn from a data stream instead of from a pre-existing static database.
  1. Because you want to keep your model very up-to-date.
  2. Because your database is too huge to fit in memory, and you don't want to read it off disk more than once.
     - Thin task is know as online learning.
- Any algorithm can be re-trained from scratch every time a new row is added from the stream.
  - E.g., MaxEnt you repeat your O(dn) training for each of n data itmes.
  - Inefficient!! Leads to n*O(dn)=$O(dn^2)$
- An algorithm that can update the model from the stream in O(1) (i.e., without revisiting the old database) has the Incremental property.

# Naïve Bayes Classifier: Online Learning

- Naïve Bayes is naturally online incremental!
- If you want to learn from a continuous stream of observations
  - Maintain your sufficient statistics $N_j$
    - (i.e., how many times each token j is associated with the current class)
  - Add +1 to the appropriate $N_j$ each new observation $x_j$
- There is corresponding regularization for the continuous version.

$$w_j \mid D = \left(N_j + \lambda\right) / \sum_j \left(N_j + \lambda\right)$$

$$w_j \mid D, D' = \left(N_j + N_j' + \lambda\right) / \sum_j \left(N_j + N'_j + \lambda\right)$$

# Naïve Bayes Classifier: Summary

- Naïve Bayes Classifier:
  - Representation: Likelihoods Bayes
  - Evaluation: Likelihood
  - Train:
    - Exact, maximum likelihood
      - (Each attribute independently)
    - Set the prior manually or ML
  - Test: Maximum A-Posteriori

$$p(H \mid D_1, D_2) = \frac{p(D_1 \mid H)\, p(D_2 \mid H)\, p(H)}{p(D_1, D_2)}$$

- Properties:
  - Train Complexity: O(dn)
  - Test Complexity: O(d)
- Good in high dimensions
  - Even d>n
- Good for Big Data
  - Incremental online
  - One-pass
- Can change priors
  - Good for mixed-type data

# Case Studies: View Angle Classification (EECS Work! ☺)

Decision tree classifies face pixels, predicts view direction.

# Classification: Overview

- Decision Trees
- Naïve Bayes
- Performance Metrics

# Performance Metrics

- Right metric to use depends on the application
  - Misuse of metric can be very misleading. So better understand them!
- Accuracy
- Confusion Matrix
- Expected Utility
- ROC Curve

# Performance Metrics:

## Accuracy

- If classifier makes predictions $y_{est}$ and the true values are $y_{tru}$
- Accuracy: Percentage of correct answers
- Advantages:
  - Easy, single number
- Limitations:
  - Doesn't account for imbalanced data:
    - E.g., Loans: 90% of people overall pay back their loan
    - Bank classifies good/bad borrowers to make lending decisions
    - If classify **all** as good => 90% "accurate" …but useless!
  - Doesn't account for which mistakes are made
  - Doesn't account for classifier calibration

$$Acc = \frac{1}{N}\sum_{i}^{N} I(y_i^{est} = y_i^{tru})$$

# Performance Metrics:
## Confusion Matrix

- The confusion matrix compares how many instances of each actual category are predicted as each estimated category.
  - The sum of the confusion matrix diagonal gives the accuracy.
    - (Accuracy = % of correct answers, 7/10 = 70% in this example)

| | | Actual | |
|---|---|---|---|
| | | 1 | 0 |
| Predicted | 1 | TP | FP |
| | 0 | FN | TN |

| | | Actual | |
|---|---|---|---|
| | | 1 | 0 |
| Predicted | 1 | 4 | 1 |
| | 0 | 2 | 3 |

# Performance Metrics:

## Confusion Matrix

- The confusion matrix compares how many instances of each actual category are predicted as each estimated category.

- Sometimes which mistakes you make matter more than the total number of mistakes
  - E.g., Loans. Predicting **g**ood/**b**ad credit
- Consider two classifier results
  - Accuracy = 50% in each case
  - Both classifiers get the bank 3 loans worth of interest payments
  - But which is more useful?
  - Classifier A: Lost business: 1, Bad Loans: 4
  - Classifier B: Lost business: 4, Bad Loans: 1

| | | Actual | |
|---|---|---|---|
| | | G | B |
| Predicted | G | 3 | 4 |
| | B | 1 | 2 |

| | | Actual | |
|---|---|---|---|
| | | G | B |
| | | 3 | 1 |
| | | 4 | 2 |

# Performance Metrics: Confusion Matrix

- Accuracy results can mislead if you have imbalanced data
  - Normalised confusion matrix can reveal this
- E.g., Assume 90% loans are good, so an accurate classifier reports 100% good loans. Overall Accuracy = 9/10 = 90%. But it's useless.
  - True Positive Rate (Frac of positives identified as positive)
  - True Negative Rate (Frac of negatives identified as negative)
  - Old diagonal: (TP+FP)/N=90%. New diagonal: (TPR+FPR)/2=50%

| Predicted \ Actual | 1 | 0 |
|---|---|---|
| 1 | TP | FP |
| 0 | FN | TN |

| Predicted \ Actual | 1 | 0 |
|---|---|---|
| 1 | TPR | FPR |
| 0 | FNR | TNR |

| Predicted \ Actual | G | B |
|---|---|---|
| G | 9 | 1 |
| B | 0 | 0 |

| Predicted \ Actual | G | B |
|---|---|---|
| G | 1.0 | 1.0 |
| B | 0 | 0 |

# Performance Metrics: Confusion Matrix

- Different applications care about different parts of the confusion matrix.
  - E.g., bank cares more about minimizing FPR (bad loans) than FNR (lost business)
  - E.g., High security system cares more about minimizing FNR (permitted breakins) than FPR (false alarms).
- Why? Because each outcome has a different cost.

| Predicted \ Actual | 1 | 0 |
|---|---|---|
| 1 | TP | FP |
| 0 | FN | TN |

| Predicted \ Actual | 1 | 0 |
|---|---|---|
| 1 | TPR | FPR |
| 0 | FNR | TNR |

| Predicted \ Actual | G | B |
|---|---|---|
| G | 3 | 1 |
| B | 4 | 2 |

| Predicted \ Actual | G | B |
|---|---|---|
| G | 3/4 | 1/4 |
| B | 4/6 | 2/6 |

# Performance Metrics: Expected Value

- Which loan classifier is better, and by how much?
  - A makes more good loans, but B makes less bad loans
  - Expected Value calculation gives a single number given a confusion matrix and cost matrix

**EV = P(Outcome1)*Val(Outcome1)+P(Outcome2)*Val(Outcome2)**

Costs

| Predicted \ Actual | G | B |
|---|---|---|
| G | $2 | -4$ |
| B | -$0.1 | -0.1$ |

| Predicted \ Actual | G | B |
|---|---|---|
| G | 3 | 4 |
| B | 1 | 2 |

EV = (2*3 - 0.1*1 - 4*4 – 0.1*2)/10
= -1$ per customer

| Predicted \ Actual | G | B |
|---|---|---|
| G | 2 | 0 |
| B | 2 | 6 |

EV = (2*2 - 0.1*2 - 0*4 - 0.1*6)/10
= 0.32$ per customer

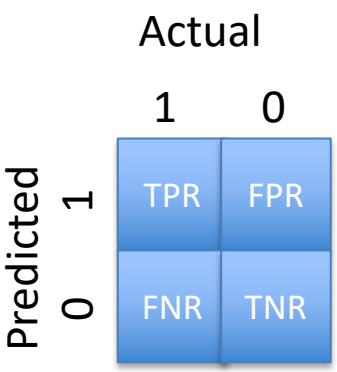
# Performance Metrics: ROC

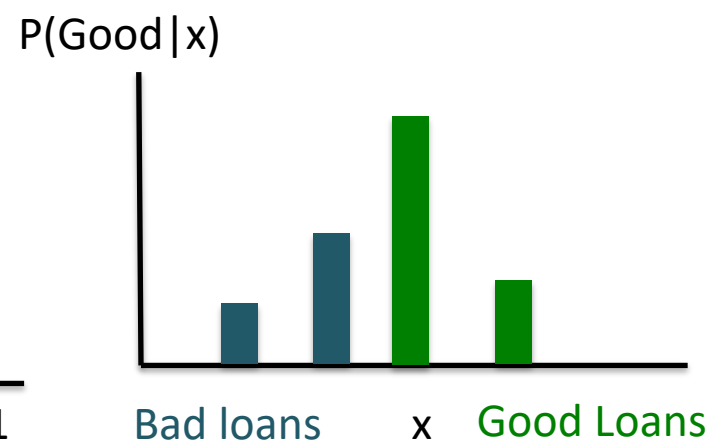
- All of the metrics discussed so far depend on classifier calibration
  - You are correct if your final estimate is $y_{tru}=y_{est}$
  - But good binary classifiers can output a confidence as well as a class…
  - By default the classifier says: $y_{est}=1$ if $p(y)>0.5$
    - If you are worried about FPs, you could say: $y_{est}=1$ if $p(y) > 0.75$
    - If you want to maximize TPs you could say: $y_{est}=1$ if $p(y) > 0.25$
- This threshold will change the distribution in the confusion matrix
  - Since threshold is user/business context dependent…..
- Is there a way to evaluate a classifier independently of the threshold?
  - So we can evaluate independently of the end user.

| | | Predicted | |
|---|---|---|---|
| | | 1 | 0 |
| Actual | 1 | TP | FN |
| | 0 | FP | TN |

# Performance Metrics: ROC

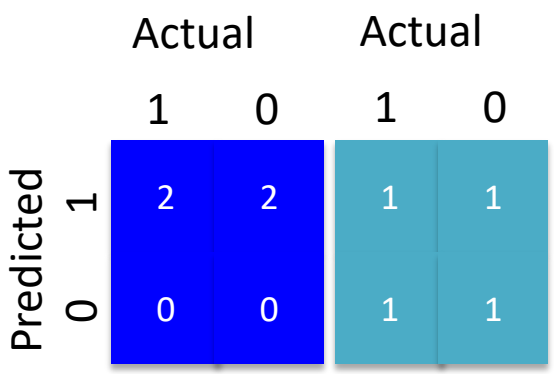
- Consider a variety of thresholds
  - Each threshold defines a TPR and FPR

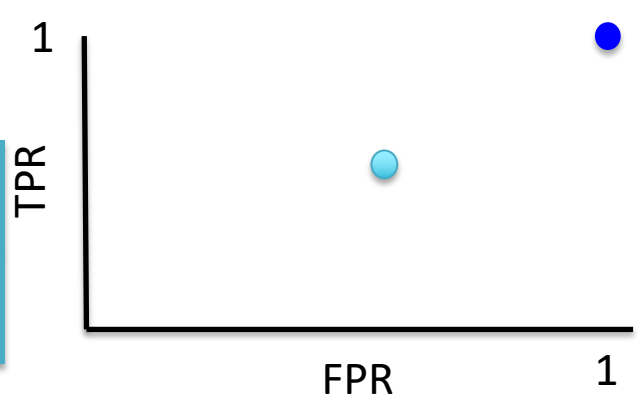


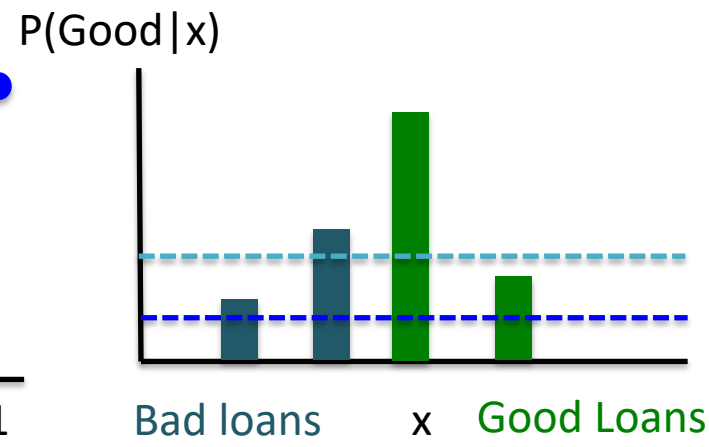


# Performance Metrics: ROC

- Consider a variety of thresholds
  - Each threshold defines a TPR and FPR
  - The ROC curve is the graph of TPRs and FPRs
    - (Receiver operating characteristic)

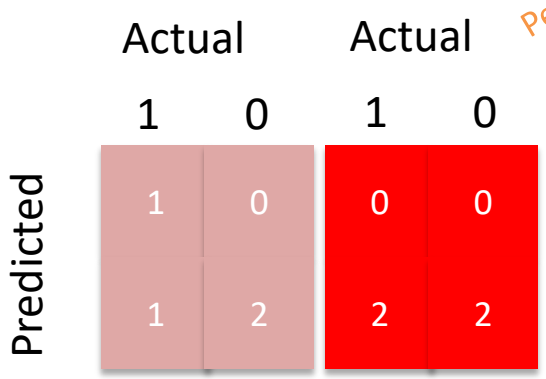


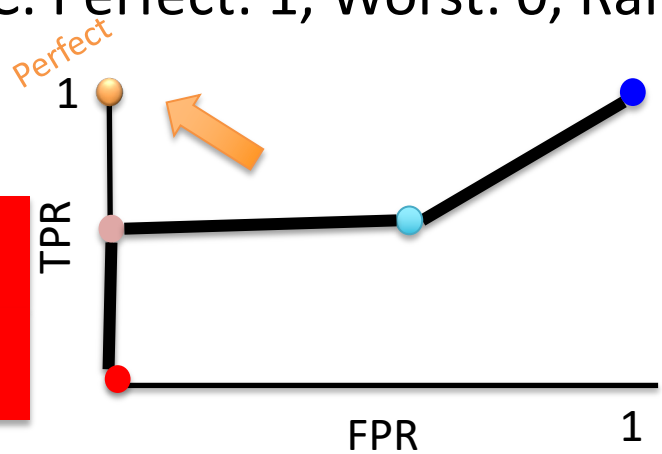



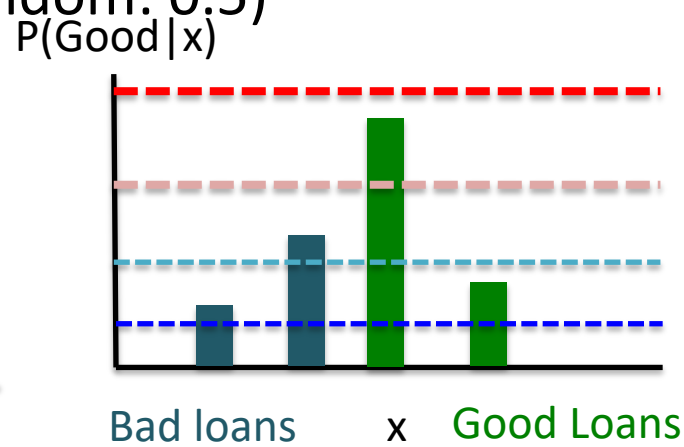



# Performance Metrics: ROC

- Consider a variety of thresholds
  - Each threshold defines a TPR and FPR
  - The ROC curve is the graph of TPRs and FPRs
    - (Receiver operating characteristic)
  - Better ROC curves approach the top left
  - Area under the ROC curve is a threshold-independent measure of goodness
    - (AUROC: Perfect: 1, Worst: 0, Random: 0.5)



# Summary

- Understand meaning and uses of different performance metrics:
  - Accuracy, confusion matrix, expected value and ROC curve

# Summary: You Should Know

- What is the process for classification using a Decision Tree?
- Sketch an algorithm to learn a Decision Tree
- What is the process for classification using Naïve Bayes
- Sketch the algorithm to learn Naïve Bayes
- Pros and cons of Decision Tress versus Naïve Bayes
- Understand meaning and uses of different performance metrics:
  - Accuracy, confusion matrix, expected value and ROC curve